



git

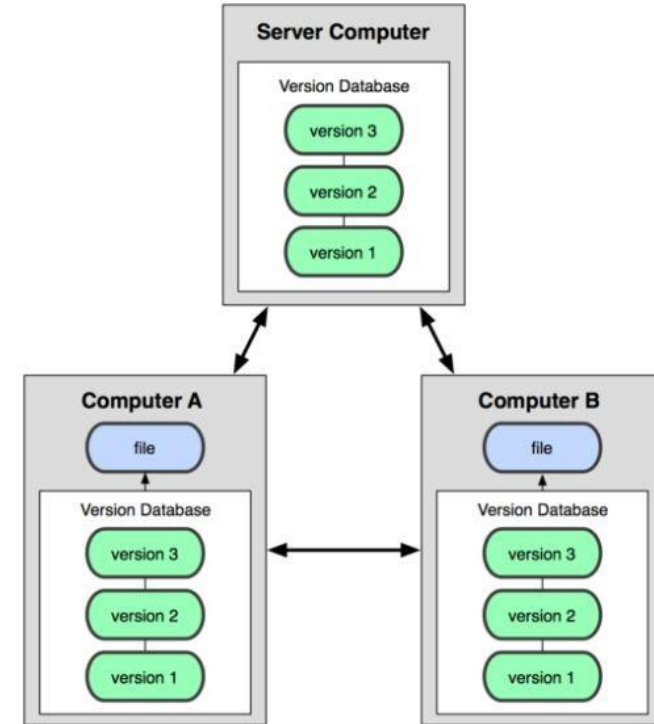


Atlassian

SourceTree

Git

- Git es un sistema distribuido de control de versiones.
- Terminología: En el lenguaje de git, una "versión" se llama "commit".
- Git mantiene un registro de la historia de tus commits, así que puedes volver atrás y mirar versiones anteriores, o simplemente abandonar la versión actual y volver a alguna versión anterior.
- Tu repositorio local es una copia completa de todo lo que hay en el servidor remoto.





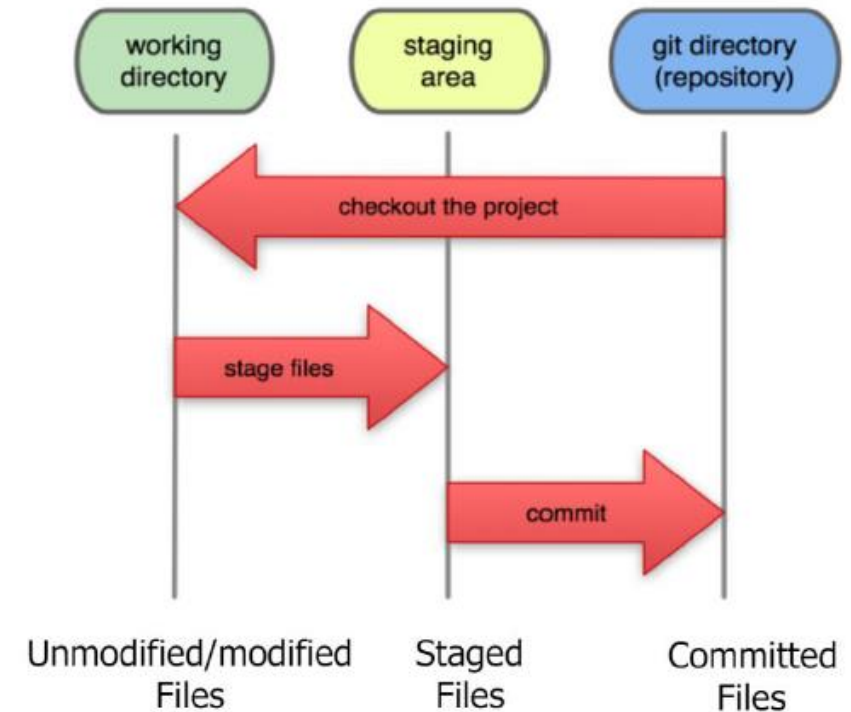
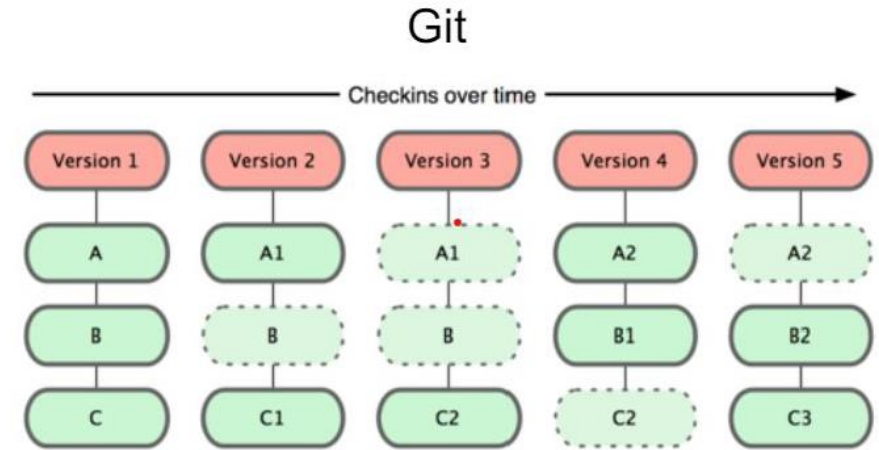
Git mantiene "instantáneas" del estado completo del proyecto.

- Algunos archivos cambian en un checkin (commit), otros no.

- Desde un punto de vista lógico, parece que Git guarda una copia completa del proyecto con cada commit. Puedes restaurar cualquier commit y obtener una copia completa del estado del repositorio en ese momento.

•Internamente :

- Git no guarda archivos completos repetidos si no han cambiado. En lugar de eso:Guarda una referencia a los archivos que no han cambiado.
- Solo almacena los archivos que han sido modificados en el nuevo commit.



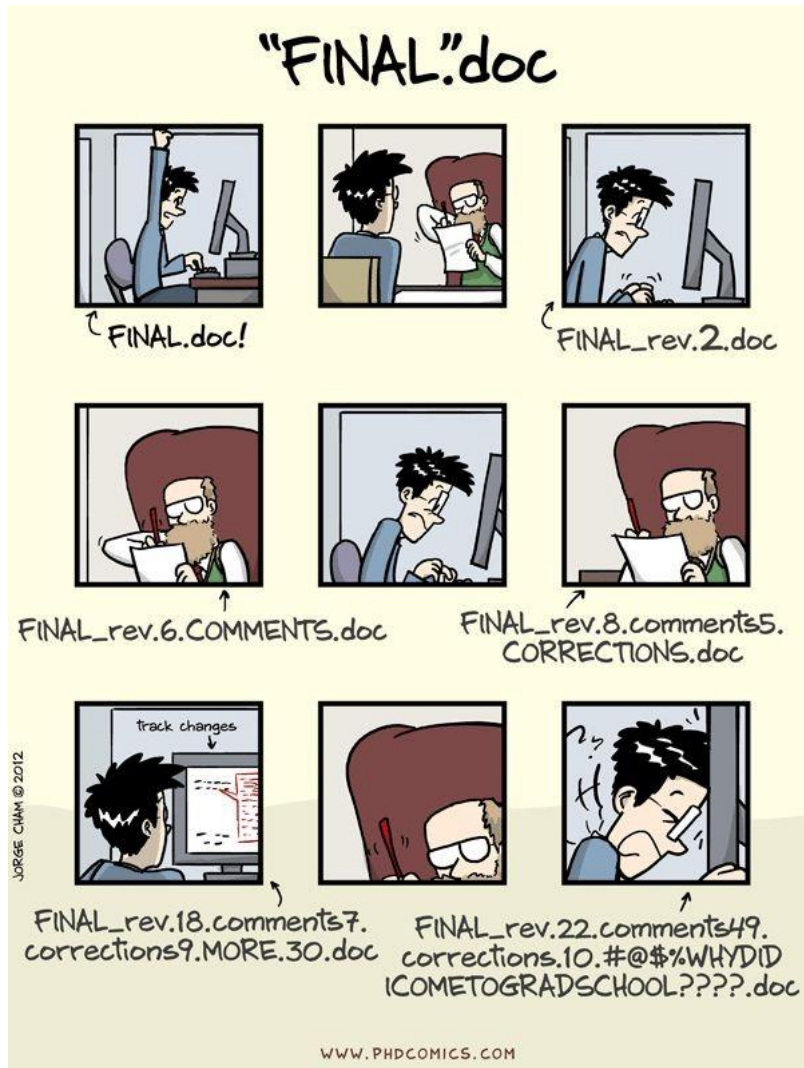


- **¿Qué es SourceTree?**

- Es una herramienta gráfica que simplifica el uso de Git, permitiendo realizar tareas comunes sin necesidad de usar comandos.

<https://www.sourcetreeapp.com/>

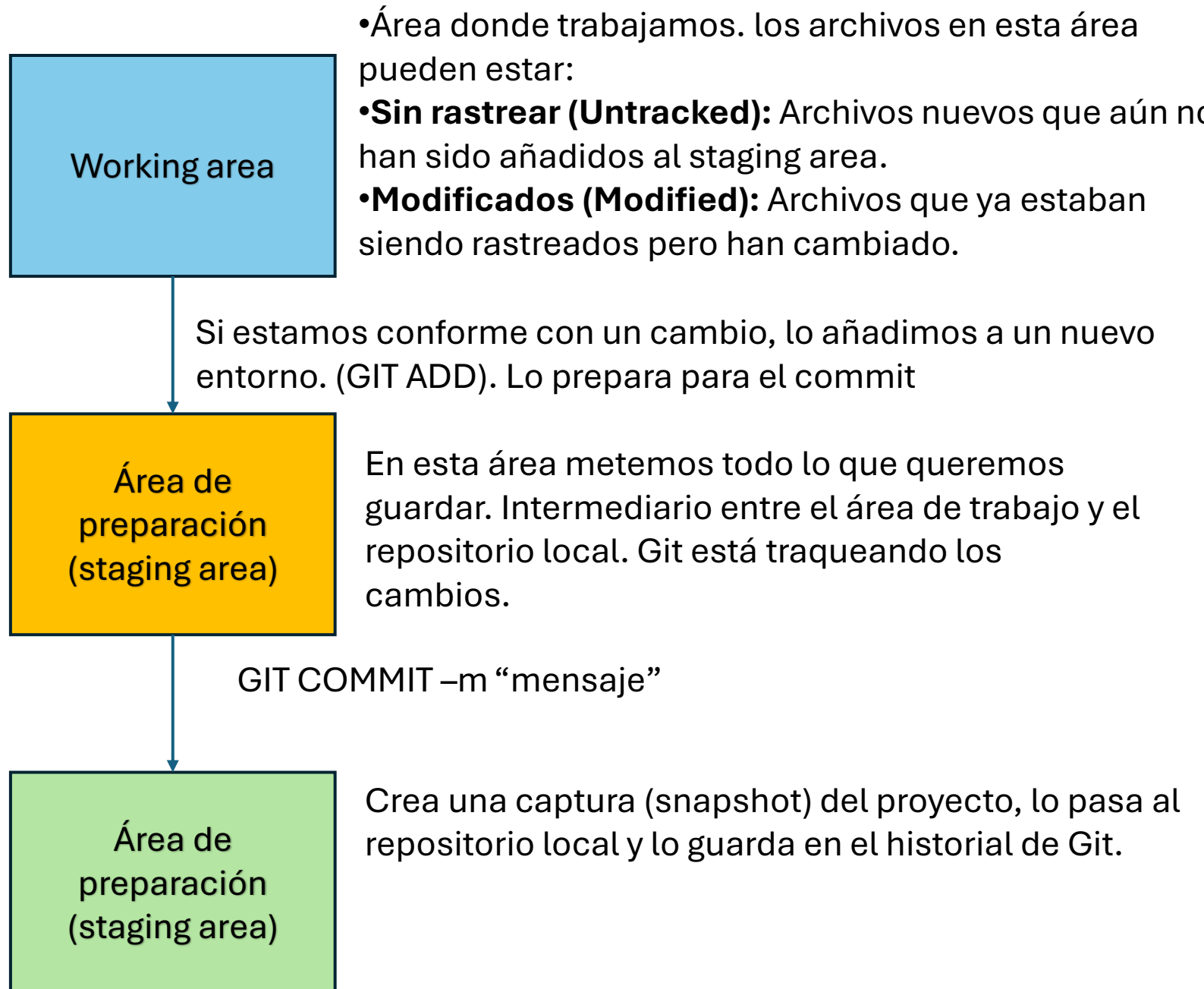
Motivación



- El desarrollo de proyectos supone un proceso iterativo de mejoras y cambios. Es necesario tener un sistema de organización y seguimiento de los mismos que, por ejemplo, permita guardar snapshots del proyecto.
- Los proyectos pueden ser llevados a cabo por múltiples personas y todas ellas pueden modificar los mismos archivos en paralelo. Es necesario implementar un modelo de branching (ramas), que permite trabajar a la vez en diferentes características (funcionalidades) y experimentar sin romper nada.
- Una vez nuestras funcionalidades pasan los test y están preparadas es necesario juntar todo nuestro trabajo. Tenemos entonces un modelo⁵ de merging, para juntar las diferentes versiones de un fichero.
- Funcionalidad de Roll-back: permite volver hacia atrás si nos equivocamos.

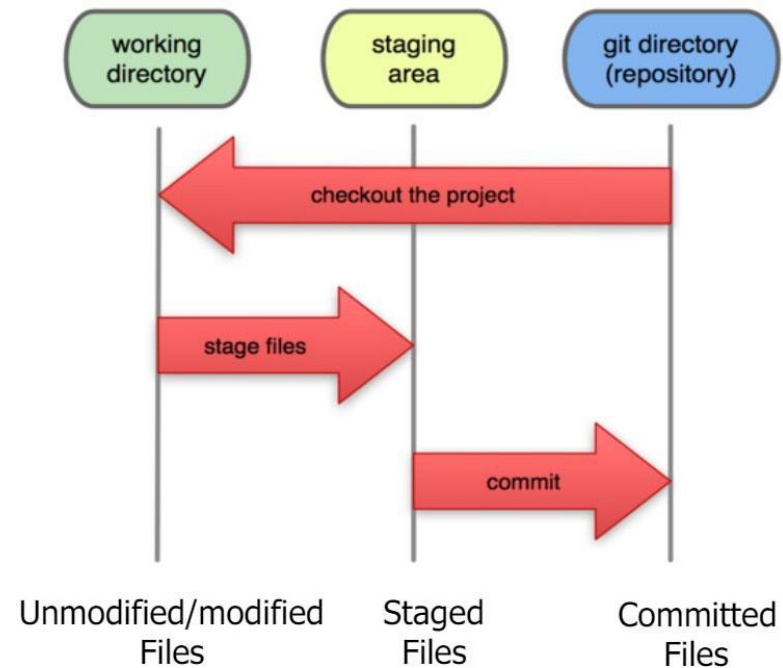
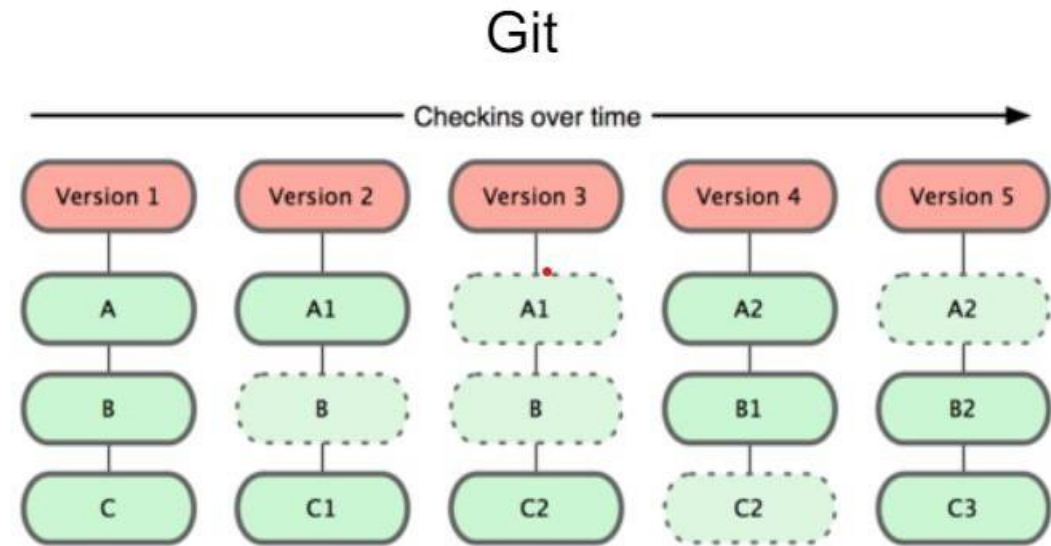
Git – Flujo de trabajo proyecto

- Git trabaja por capas.



Git

- Git mantiene "instantáneas" del del estado completo del proyecto.
- Cada versión checkin del código global tiene una copia de de cada archivo.
- Algunos archivos cambian en un checkin, otros no.
- Más redundancia, pero más rápido



Git – Ramas

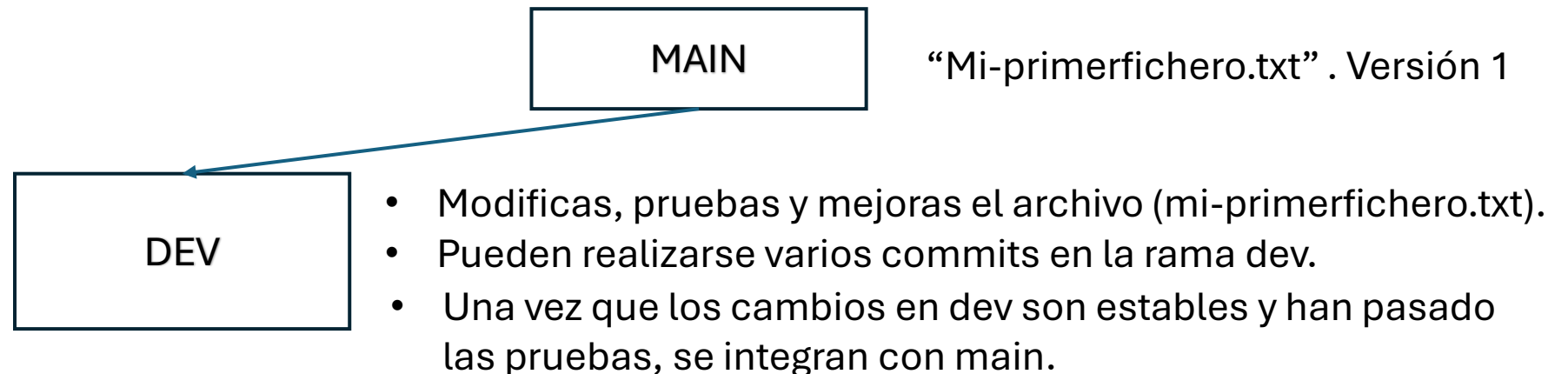
Una rama es una línea independiente de desarrollo dentro de un repositorio. Permite trabajar en nuevas funcionalidades, corregir errores o experimentar sin afectar la versión principal del proyecto.

Rama MAIN : Contiene el código estable del proyecto. Actúa como la versión “oficial” del proyecto. La versión que se sube a producción.

- Regla general : No se realizan cambios directos en main a menos que estén completamente probados

Rama “Dev”. Es una copia exacta de la rama Main. Es donde se desarrollan o prueban nuevas funcionalidades, arreglos o experimentos.

- Los cambios realizados en dev no afectan a main hasta que se fusionen explícitamente (merge).



Git – Ramas

- **TERMINAL :**

- **Crear la rama dev:**

- **Esta rama se basa en la rama en la que te encuentres (por ejemplo main)**

- `git checkout -b dev`

- Esto crea una rama nueva llamada dev basada en main.

- **Hacer cambios en dev:**

- Editas el archivo mi-primerfichero.txt.

- Preparas los cambios:

- `git add mi-primerfichero.txt`

- Confirmas los cambios

- `git commit -m "Mejorar mi-primerfichero.txt"`

- **Fusionar dev con main:**

- Cambias a la rama main:

- `git checkout main`

- Fusionas los cambios:

- `git merge dev`

Git

Merge

Es la acción de combinar los cambios de una rama en otra.

Ejemplo: Si terminaste de trabajar en nueva-funcionalidad, puedes fusionar (merge) esa rama con la rama principal (main) para que los cambios queden integrados.

Merge con conflicto: Si ambas ramas tienen cambios en los mismos archivos, Git pedirá que resuelvas los conflictos manualmente.

```
git merge <nombre_de_la_rama>
```

Git

Push

Es el proceso de enviar los cambios desde tu repositorio local a un repositorio remoto (como GitHub).

•**Ejemplo:** Has trabajado en tu máquina local, y ahora quieres compartir tus cambios con el equipo o guardarlos en GitHub.

Flujo:

1. Haces cambios y los confirmas (commits).
2. Envías esos cambios al servidor remoto con git push.

`git push origin <nombre_de_la_rama>.`

Git

Pull

Es el proceso de traer cambios del repositorio remoto a tu repositorio local y fusionarlos automáticamente.

- Ejemplo:** Tu compañero ha subido nuevos cambios al repositorio remoto. Usas git pull para actualizar tu copia local con esos cambios.

Flujo:

- 1.Descarga los cambios del remoto.
- 2.Fusiona esos cambios en tu rama local.

•Comando:

- git pull origin <nombre_de_la_rama>.

Git

FETCH

git fetch es un comando de Git que permite **descargar cambios** de un repositorio remoto (como GitHub) a tu repositorio local, pero **sin fusionarlos automáticamente** con tu rama actual.

Diferencia con PULL

- **git fetch**: Solo trae los cambios desde el repositorio remoto a tu copia local, pero los guarda en una zona especial llamada "origen remoto". No afecta ni combina nada en tu rama de trabajo.
- **git pull**: Trae los cambios del remoto **y además los fusiona** directamente con tu rama actual.

- Descargar los cambios : `git fetch origin`
- Ver qué ha cambiado: `git log HEAD..origin/main`
- Fusionar los cambios: `git merge origin/main`



Iniciando un proyecto de IA Creación del repositorio GIT y reglas sobre las ramas

Ejercicio 1: Configuración inicial de Git

Objetivo

Configurar Git con tu nombre y correo electrónico para identificar quién hace los cambios.

Instrucciones

1. Abrir SourceTree.

2. Ve a **Tools (Herramientas) → Options (Opciones)** y configura:

1. **Full Name:** Escribe tu nombre completo (por ejemplo, "Ana García").

2. **Email Address:** Introduce tu correo (por ejemplo, "ana.garcia@gmail.com").

3. **Verificar configuración desde la terminal de Git Bash o PowerShell**

```
git config --global user.name
```

```
git config --global user.email
```

Ejercicio 2: Crear un repositorio local

Objetivo

Aprender a inicializar un repositorio Git local, hacer un commit y trabajar con SourceTree.

Instrucciones

1. Crea una carpeta en tu ordenador llamada mi-proyecto.
2. Abre SourceTree y selecciona **File → New/Clone → Create**.
 - Haz clic en Browse y selecciona la carpeta mi-proyecto donde quieres crear tu repositorio.
 - Haz clic en Create. Esto inicializará un repositorio en la carpeta existente.
3. Crea un archivo README.md dentro de la carpeta **mi-proyecto** y escribe:
Mi Proyecto
Este es mi primer repositorio Git
4. En SourceTree:
 - Ve a la pestaña **File Status**.
 - Selecciona **Stage All** para preparar el archivo.
 - Haz clic en **Commit** y escribe un mensaje: **feat:** añade README inicial.

Resultado esperado

El repositorio tendrá un commit inicial visible en la pestaña **History**.

Ejercicio 2: Crear un repositorio local

El archivo **README.md** es importante porque describe tu proyecto. Puedes incluir:

- **Título del proyecto**
- **Descripción breve.**
- **Instrucciones de instalación.**
- **Cómo usar el proyecto.**
- **Autores o colaboradores.**

Convenciones de Mensajes de Commit

- **feat:** Nueva funcionalidad o característica.
- **fix:** Corrección de errores.
- **docs:** Cambios en la documentación.
- **style:** Cambios que no afectan la lógica del código (formato, espacios, etc.).
- **refactor:** Reestructuración del código sin cambiar su funcionalidad.
- **test:** Añadir o modificar pruebas.
- **chore:** tareas menores que **no afectan directamente el código de la aplicación o su funcionalidad.**

Ejercicio 2.1: Gestión de cambios

Objetivo

Practicar la creación de múltiples archivos y commits.

Aprender a manejar el área de **staging** (preparar cambios) y confirmar cambios específicos.

Instrucciones

1. Añadir un nuevo archivo al repositorio local:

- En la carpeta de tu proyecto (mi-proyecto), crea un archivo llamado introduccion.txt y escribe el siguiente contenido:

Este es un proyecto de ejemplo para aprender Git.

- En SourceTree, ve a la pestaña **File Status** y observa que el archivo aparece como un cambio no rastreado (**Unstaged**).

2. Realizar el primer commit del nuevo archivo:

- Haz clic en Stage All para añadir el archivo al área de staging.
- Luego, haz clic en Commit y escribe un mensaje como:
feat: añade archivo introduccion.txt inicial.

Ejercicio 2.1: Gestión de cambios

3. Modificar introducción.txt

- Edita el archivo **introduccion.txt** y añade una nueva línea al final.

“Esta es la primera actualización de este archivo”
- En SourceTree, seleccionas el archivo en la pestaña **File Status** y haces clic en **Stage** para moverlo al área de staging (prepararlo para el commit).
- Te das cuenta de que no quieres incluir este archivo en el commit por ahora. Entonces:
 - Selecciona el archivo introducción.txt en la lista Staged Files (archivos preparados)
- Haz clic en Unstage
- El archivo vuelve a la lista de cambios **Unstaged**, pero los cambios en el archivo permanecen intactos.
- Puedes decidir:
 1. No incluirlo en este commit y dejarlo para más tarde.
 2. Volver a moverlo al área de staging (**Stage**) y luego hacer el commit.

Ejercicio 2.1: Gestión de cambios

4. Crea otro archivo llamado **detalles.md** y escribe el siguiente contenido

Detalles del Proyecto

- Este proyecto usa Git para controlar versiones.
 - Usaremos comandos como fetch, pull y merge.
- Ambos cambios aparecerán en la pestaña **File Status** de SourceTree.

5. **Seleccionar cambios para commits separados:**

- Añade solo el archivo **detalles.md** al área de staging y haz un commit con el mensaje:
feat: añade archivo detalles.md con información del proyecto.

Ejercicio 3: Archivo .gitignore

En el archivo .gitignore defines patrones de archivos o carpetas que quieres que Git ignore.

El archivo .gitignore debe de estar en la raíz del repositorio (o en las subcarpetas específicas si quieres reglas más precisas)

Ejemplos de contenido del archivo .gitignore:

Archivos temporales

*.log

*.tmp

Carpetas específicas

/node_modules/

/dist/

Archivos sensibles

secret.txt

.env

Ejercicio 3: Archivo .gitignore

El archivo **.env** es un archivo de texto utilizado para almacenar **variables de entorno** en proyectos de desarrollo. Estas variables son datos que tu aplicación necesita para funcionar, pero que no deberían incluirse directamente en el código fuente porque pueden contener información sensible o específica del entorno en el que se ejecuta la aplicación.

¿Qué se guarda en un archivo .env?

1.Datos sensibles:

- Claves API.
- Tokens de autenticación.
- Contraseñas de bases de datos.

2.Configuraciones específicas del entorno:

- URL de bases de datos (producción, desarrollo o pruebas).
- Puertos en los que se ejecuta la aplicación.
- Variables de configuración (por ejemplo, habilitar o deshabilitar logs).

Ejercicio 3: Archivo .gitignore

1. Crear un archivo .gitignore.

- En la carpeta del proyecto, crea un archivo llamado .gitignore
- Crea un archivo Nuevo → Documento de texto.
- Nómbralo como .gitignore (asegúrate de eliminar la extensión .txt)
- Dentro del archivo, escribe las reglas que desees ignorar. Ejemplo:

*.log

- Crea un archivo en la carpeta del proyecto que coincida con el patrón y con cualquier contenido
 - Errores.log
- Ve a SourceTree y verifica que errores.log no aparece en la pestaña File Satus
- Aunque .gitignore no esté confirmado y esté en la zona de Unstaged files, debe de funcionar.
- Prepara (staged) el archivo .gitignore
- Haz un commit para confirmar .gitignore y escribe un mensaje
 - chore:** añade .gitignore para gestionar archivos ignorados
- Verifica el Historial :
 - Ve a la pestaña History y verifica que el archivo .gitignore ha sido confirmado

Ejercicio 4: Conectar repositorios locales con GitHub

Conectar un repositorio local a GitHub

1. En GitHub, crea un repositorio vacío llamado mi-proyecto.

¿Debe ser público o privado?

Cuando creas un repositorio en GitHub, tienes dos opciones:

•Público:

- Cualquiera en Internet puede ver el contenido del repositorio, pero solo tú (o los colaboradores que agregues) pueden hacer cambios.
- Útil si estás trabajando en un proyecto que no necesita ser secreto.

•Privado:

- Solo tú y los colaboradores que agregues podrán ver y trabajar en el repositorio.
- Útil para proyectos que contienen información sensible o que quieres mantener en secreto.

Para este ejercicio debería ser un repositorio privado, **pero lo vamos a hacer público**. Si lo hacemos privado y queremos realizar reglas para las ramas, tenemos que actualizar a una versión de pago.

Ejercicio 4: Conectar repositorios locales con GitHub

Conectar un repositorio local a GitHub

¿Hay que crear un archivo README.md en GitHub?

NO lo crees en GitHub si ya lo hiciste en local:

- Si en el repositorio local ya has creado un archivo README.md, no lo vuelvas a crear en GitHub, ya que esto puede generar conflictos cuando intentes conectarlo y subir los archivos.
- Tu archivo local README.md tiene contenido y el archivo en GitHub estará vacío, lo que podría causar problemas al hacer el primer **Push**.

Ejercicio 4: Conectar repositorios locales con GitHub

Crea un repositorio vacío en GitHub:

- Ve a [GitHub](#) e inicia sesión.
- Haz clic en el botón **New** (Nuevo) o en el símbolo "+" → **New repository**.
- Introduce el nombre: [mi-proyecto](#).
- Selecciona PÚBLICO**
- No marques las opciones de inicialización como**
 - Add a README file, .gitignore, o licencia. Esto deja el repositorio vacío.
- Haz clic en **Create repository**.

Ejercicio 4: Conectar repositorios locales con GitHub

Abre SourceTree y configura el remoto:

- Ve a **Repository** → **Repository Settings** → **Remotes** → **Add**
- En la ventana que se abre:
 - Remote Name:** Escribe `origin` (es solo un nombre para identificar el remoto. Es una convención standard)
 - URL:** Copia y pega la URL HTTPS del repositorio que aparece en GitHub
 - Haz clic en **OK**.
- Sube tus commits al repositorio remoto:**
 - Haz clic en **Push** en SourceTree.
 - Selecciona la rama principal (main)
 - Confirma y espera a que los cambios se suban.
 - Verifica en GitHub que tus archivos, incluido README.md, aparecen en el repositorio remoto.

Ejercicio 4: Conectar repositorios locales con GitHub

Abre SourceTree y configura el remoto:

- Ve a **Repository** → **Repository Settings** → **Remotes** → **Add**
- En la ventana que se abre:
 - **Remote Name:** Escribe `origin` (es solo un nombre para identificar el remoto. Es una convención standard)
 - **URL:** Copia y pega la URL HTTPS del repositorio que aparece en GitHub
 - Haz clic en **OK**.
- **Sube tus commits al repositorio remoto:**
 - Haz clic en **Push** en SourceTree.
 - Selecciona la rama principal (main)
 - Confirma y espera a que los cambios se suban.
 - Verifica en GitHub que tus archivos, incluido README.md, aparecen en el repositorio remoto.

Ejercicio 5: Clonar un repositorio desde Github

La opción de clonar se utiliza para copiar un repositorio completo desde un servidor remoto (como GitHub) a tu máquina local.

- Incluye todos los archivos, el historial de commits y la configuración del repositorio remoto.

Cuándo se usa:

- Es útil para trabajar en proyectos ya existentes o para iniciar un proyecto colaborativo con otros usuarios.

Ejercicio 5: Clonar un repositorio desde Github

Paso 1: Crear un repositorio en GitHub

1. Ve a GitHub e inicia sesión.
2. Haz clic en el botón **New Repository** (Nuevo repositorio) o en el símbolo "+" → **New repository**.
3. Configura el repositorio:
 - **Nombre del repositorio:** proyecto-clonado.
 - Marca la opción **Add a README file** para incluir un archivo README.md inicial.
 - Deja las demás opciones por defecto y haz clic en **Create Repository**.

Ejercicio 5: Clonar un repositorio desde Github

Paso 2: Clonar el repositorio a tu ordenador

1. Obtén la URL del repositorio:

- En la página de tu repositorio en GitHub,
 - haz clic en el botón **Code** →
 - selecciona la opción **HTTPS** → copia la URL (por ejemplo: `https://github.com/tuusuario/proyecto-clonado.git`).

2. Abre SourceTree y clona el repositorio:

- Haz clic en el botón File -> **Clone**.
- En el campo **Source Path / URL**, pega la URL copiada del repositorio.
- En el campo **Destination Path**, selecciona una carpeta en tu ordenador donde quieres que se guarde el proyecto.
- Haz clic en **Clone**.

Ejercicio 5: Clonar un repositorio desde Github

Paso 2: Clonar el repositorio a tu ordenador

3.Verifica el contenido clonado:

- Ve a la carpeta local donde clonaste el repositorio.
- Verifica que el archivo README.md está presente.

Configura un **token de acceso personal (PAT)** como tu contraseña en lugar de una contraseña habitual. Puedes generar uno en [GitHub Settings → Developer Settings → Personal Access Tokens](#).

<https://github.com/settings/tokens>

Ejercicio 6: Configuración inicial de ramas en GitHub

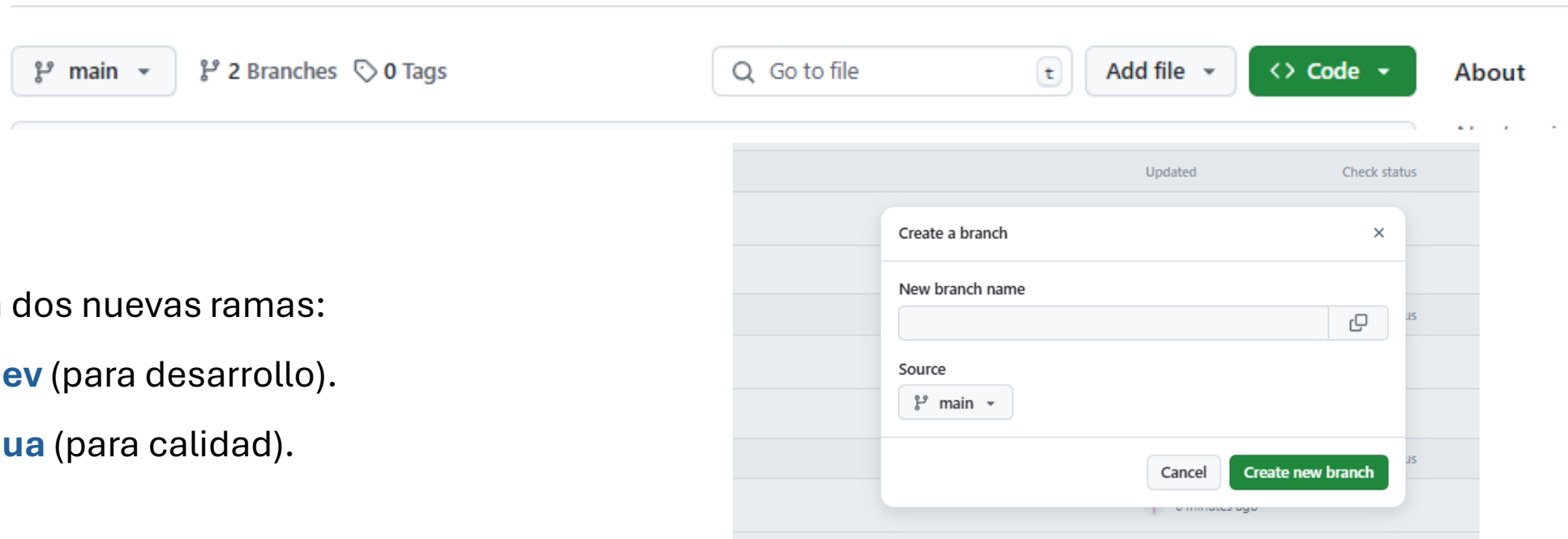
Objetivo

Configurar las ramas principales en el repositorio de GitHub para simular un flujo de trabajo.

Instrucciones

1. Ve a tu repositorio en GitHub (mi-proyecto).
2. Haz clic en Branches . Y luego **New branch**

3. Crea dos nuevas ramas:
 - **dev** (para desarrollo).
 - **qua** (para calidad).



Ejercicio 6: Configuración inicial de ramas en GitHub

1. Rama dev (Desarrollo)

Es el entorno donde se desarrollan nuevas funcionalidades o se corrigen errores. Es la rama donde los desarrolladores trabajan.

- **Características:**

- Aquí es donde se puede "romper" el código sin problema, ya que está pensado para desarrollar y probar cambios iniciales.
- Se realizan pruebas preliminares para asegurarse de que las funcionalidades funcionan como deberían.

- **Objetivo:** Garantizar que los desarrollos básicos funcionan antes de moverlos a la siguiente etapa.

Ejercicio 6: Configuración inicial de ramas en GitHub

2. Rama qa (Calidad)

- **Propósito:** Es un entorno de pruebas más cercano a producción. Aquí se evalúan los cambios realizados en dev para comprobar que todo funciona correctamente antes de desplegar.
- **Características:**
 - Es un entorno con recursos más realistas, similar al entorno de producción.
 - Permite realizar pruebas avanzadas para asegurarse de que el código no tiene errores y que el sistema responde como se espera.
- **Objetivo:** Validar que los cambios son estables y funcionales, como si ya estuvieran en producción.

Ejercicio 6: Configuración inicial de ramas en GitHub

3. Rama main (Producción)- Es la rama por defecto

- Propósito:** Es la rama donde se despliega el servicio final que utilizarán los clientes. Solo se fusionan cambios completamente validados desde qa.
- Características:**
 - Aquí no se realizan cambios directos, solo se aceptan los cambios provenientes de qa que hayan sido verificados como correctos.
 - Se encarga de desplegar servicios, como modelos de machine learning, páginas web o cualquier otro endpoint al que acceden los clientes.
- Objetivo:** Asegurar que los clientes tienen acceso únicamente al producto final completamente funcional y sin errores.

Ejercicio 6: Configuración inicial de ramas en GitHub

1. Flujo de trabajo establecido :

- **Desarrollo en dev:** Todo cambio o nueva funcionalidad comienza en la rama dev.
- **Pruebas en qa:** Una vez que los desarrollos en dev están terminados y comprobados, se fusionan con la rama qa. Esta rama se utiliza para simular un entorno lo más cercano posible a producción y realizar pruebas finales.
- **Producción en main:** Solo después de que todo funcione perfectamente en qa, los cambios se pasan a la rama main, que es la rama de producción. Aquí es donde el cliente final tiene acceso.

Propósito de este flujo:

- **Seguridad:** Evitar que cambios inestables lleguen a producción.
- **Control:** Asegurar que todas las funcionalidades estén probadas y aprobadas antes de llegar al cliente final.
- **Colaboración:** Facilitar un flujo organizado entre desarrolladores y testers.

Ejercicio 6: Configuración inicial de ramas en GitHub

2. Reglas que se van a establecer para las ramas:

- No se permite modificar qa o main directamente: Todos los cambios deben pasar primero por dev y luego progresar hacia qa y finalmente main.
- Esto garantiza que cada etapa sea revisada y aprobada antes de impactar en el producto final.

Ejercicio 6: Configuración inicial de ramas en GitHub

Reglas para las ramas . Protección de ramas y flujo de trabajo organizado

En los proyectos de desarrollo, es fundamental proteger las ramas para evitar problemas y mantener un flujo de trabajo organizado.

Proteger una rama significa aplicar una serie de **reglas** que restringen ciertas acciones para garantizar la estabilidad del proyecto.

Conceptos clave

1. Proteger ramas:

- Las reglas de protección aseguran que no se puedan realizar cambios directamente en ramas importantes como `qa` o `main` sin un proceso controlado.
- Esto evita errores accidentales o cambios que puedan romper el sistema.

Ejercicio 6: Configuración inicial de ramas en GitHub

Configurar las reglas de protección en las ramas en GitHub

1.Settings-> Rules -> Rulesets-> New rulesets -> New Branch ruleset

The screenshot shows the GitHub Settings page for a repository. The left sidebar contains navigation links: General, Access (Collaborators, Moderation options), Code and automation (Branches, Tags, Rules, Rulesets, Actions, Webhooks, Environments, Codespaces, Pages), Security (Code security, Deploy keys, Secrets and variables), and Integrations (GitHub Apps, Email notifications). The 'Rulesets' link is highlighted. The main content area is titled 'Rulesets / New branch ruleset'. It includes a 'Ruleset Name' field with the value 'no direct merge', an 'Enforcement status' dropdown set to 'Active', and a 'Bypass list' section with an 'Add bypass' button. Below the bypass list is a message 'Bypass list is empty'. The 'Targets' section is titled 'Which branches do you want to make a ruleset for?' and contains a 'Target branches' section with a description: 'Branch targeting determines which branches will be protected by this ruleset. Use inclusion patterns to expand the list of branches under this ruleset. Use exclusion patterns to exclude branches.' Below this is a 'Branch targeting criteria' table with two rows: 'Default' and 'qa', each with an 'Add target' button and a trash icon.

Branch targeting criteria		Add target
+	Default	🗑️
+	qa	🗑️

Rules

Which rules should be applied?

Branch rules

- ☐ Restrict creations
Only allow users with bypass permission to create matching refs.
 - ☐ Restrict updates
Only allow users with bypass permission to update matching refs.
 - ☒ Restrict deletions
Only allow users with bypass permissions to delete matching refs.
 - ☐ Require linear history
Prevent merge commits from being pushed to matching refs.
 - ☐ Require deployments to succeed
Choose which environments must be successfully deployed to before refs can be pushed into a ref that matches this rule.
 - ☐ Require signed commits
Commits pushed to matching refs must have verified signatures.
 - ☒ Require a pull request before merging
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
- Hide additional settings ^
- Required approvals**
- 0
- The number of approving reviews that are required before a pull request can be merged.
- ☐ Dismiss stale pull request approvals when new commits are pushed

Ejercicio 6: Configuración inicial de ramas en GitHub

Configurar las reglas de protección en las ramas en GitHub

1. Nombre de la regla:

No Direct Merge: Esta regla evitará que se realicen cambios directos en ciertas ramas protegidas. En este ejercicio estas ramas serán la rama qa, y la rama main

2. Bypass list o excepciones:

En este caso **no permitiremos excepciones**, es decir, nadie podrá saltarse las reglas.

3. Ramas afectadas (Target Branches):

- **main** (rama por defecto).
- **qa** (rama para pruebas).

Ejercicio 6: Configuración inicial de ramas en GitHub

Configurar las reglas de protección en las ramas en GitHub

4. Reglas aplicadas:

- Restrict Deletion:** Nadie podrá eliminar las ramas **main** y **qa**.
- Require a Pull Request Before Merging:** Para realizar cambios en **qa** o en **dev**, **no se podrán hacer commits directos**.

En lugar de esto, será obligatorio crear un **Pull Request** (PR). Un PR es una solicitud formal en Git para realizar un cambio en el código.

Ejercicio 6: Configuración inicial de ramas en GitHub

Configurar las reglas de protección en las ramas en GitHub

Required approvals:

En este caso, configuramos que no se necesitan aprobaciones (Required approvals = 0), ya que en este ejercicio estás trabajando solo/sola. Uno/una mismo/misma no se puede aprobar el código. Estás solo/a haciendo pruebas y te bloquearías a ti mismo/a

- En equipos: Si trabajáramos en un equipo de 3 o 4 personas, podríamos configurar que al menos **una persona debe aprobar los cambios** (Required approvals = 1). Eso quiere decir que al menos un compañero/a tiene que aprobar tus cambios. Esto fomenta una **revisión del código (Code review)** para garantizar **calidad del código y evitar errores**. Además permite **identificar errores o bugs antes de que lleguen a producción**.

Ejercicio 7: Actualización del repositorio local con nuevas ramas y reglas

•SourceTree:

1. Abre **SourceTree** y selecciona tu proyecto local (mi-proyecto) que ya está vinculado al repositorio remoto en GitHub.
2. En tu repositorio local, las ramas nuevas (**qa** y **dev**) y las reglas de protección no están presentes porque todavía no se han descargado.
3. **¿Qué comando usar para sincronizar? (Pull o Fetch)**
 - **Fetch te da el control para decidir si quieres o no combinar** los cambios remotos con los locales. Esto es útil porque puedes revisar los cambios antes de aplicarlos.
 - **Pull** hace un **Fetch** para descargar los cambios del remoto **y también los aplica automáticamente** a tu rama actual, intentando combinarlos.

Ejercicio 7: Actualización del repositorio local con nuevas ramas y reglas

¿Cuál es mejor Fetch o Pull?

• Usa fetch si:

- Quieres más control sobre cuándo integrar los cambios remotos.
- Estás en medio de un trabajo y no quieres que los cambios remotos interfieran con tu progreso.
- Recomendación : si estás trabajando en un entorno colaborativo: haz primero un fetch para revisar los cambios . Si está todo bien y estás listo/a para integrar los cambios, haz un merge.
- Esto te da más control .

• Usa pull si:

- Quieres ahorrar tiempo y estás seguro de que tu rama local está lista para integrar los cambios remotos.
- Acabas de empezar a trabajar en la rama y no tienes cambios sin confirmar.

Ejercicio 7: Actualización del repositorio local con nuevas ramas y reglas

•SourceTree:

1. Haz clic en el botón **Fetch**:

- Descarga los cambios del repositorio remoto (nuevas ramas, commits, etc.) al repositorio local, pero **NO modifica tu rama actual**.
- Las actualizaciones del remoto están disponibles en “origin/<rama>”.

2. Resultados esperados después del Fetch:

- En la sección de **Ramas** (branches) de SourceTree, verifica que ahora aparecen las ramas **qa** y **dev** en la lista.
- Estas ramas estarán listadas bajo el encabezado de **Remote** o **Ramas remotas** (Remote Branches).

3. Comprueba que están las 3 ramas

Nota importante:

En este ejercicio, **no es necesario hacer un merge** porque el objetivo es únicamente sincronizar el repositorio local con el remoto para que las ramas nuevas sean visibles.

El **merge** solo sería necesario si quisieras combinar los cambios remotos con los de tu rama local, lo cual no aplica en este caso ya que no ha habido cambios.

Ejercicio 8: Comprobar que la regla de protección funciona

Objetivo del ejercicio:

Probar que las reglas de protección configuradas para **la rama qa** impiden realizar cambios directos, evitando errores comunes como trabajar en la rama equivocada.

Instrucciones paso a paso:

1. Cambiar a la rama protegida (qa):

En **SourceTree**, hay dos formas de cambiar a la rama qa:

- **Opción 1:** Haz **doble clic en la rama qa** en la lista de ramas.
- **Opción 2:** Haz **clic derecho** en la rama qa y selecciona **Checkout**.

2. Crear un archivo de prueba:

- En tu repositorio local, crea un archivo nuevo llamado Prueba.txt.
 - Por ejemplo, abre un editor de texto, escribe algo breve, y guarda el archivo como Prueba.txt en la carpeta de tu repositorio (mi-proyecto).

Ejercicio 8: Comprobar que la regla de protección funciona

Instrucciones paso a paso:

3. Verificar cambios detectados por SourceTree:

- Regresa a SourceTree y observa que el archivo Prueba.txt aparece en la lista de **Unstaged files**.
- Esto significa que SourceTree ha detectado el archivo recién creado.

4. Intentar cometer un error común:

- **Paso 1:** Mueve el archivo Prueba.txt a la sección **Staged files** haciendo clic en el botón correspondiente.
- **Paso 2:** Haz un **commit** del archivo. Escribe un mensaje como "Prueba de reglas" y confirma el commit.
- **Paso 3:** Intenta hacer un **push** para subir el cambio al repositorio remoto.

5. Observar el resultado:

- Git y SourceTree deberían bloquear el intento de **push** directo debido a las reglas de protección configuradas en la rama qa. Esto demuestra que las reglas están funcionando correctamente.

Ejercicio 9: Actualizar el repositorio local con Push

Objetivo: Practicar cómo traer cambios remotos al repositorio local respetando las reglas.

Instrucciones:

1. En GitHub:

- Abre la rama dev del repositorio mi-proyecto
- Edita el archivo README.md desde el navegador y añade:

Cambios realizados desde GitHub en la rama dev

- Haz clic en **Commit changes** para guardar los cambios. Con el mensaje “actualizar Readme desde Github”

2. En SourceTree:

- Cambia a la rama dev.
- Haz clic en **Fetch**. Esto actualizará tu vista con los cambios disponibles sin aplicarlos aún. Te permitirá chequear los cambios.

Ejercicio 9: Actualizar el repositorio local con Push

Objetivo: Practicar cómo traer cambios remotos al repositorio local respetando las reglas.

Instrucciones:

2. En SourceTree:

- Luego, haz clic en el botón **Pull**, esto traerá los cambios remotos a tu repositorio actual

3. Realiza un cambio adicional en README.md desde tu repositorio local:

- Añade una línea como: Cambios realizados desde el repositorio local
- Haz un **commit** con el mensaje: feat: cambios locales en dev
- Haz un **push** al remoto.

Ejercicio 9: Actualizar el repositorio local con Push

Objetivo: Practicar cómo traer cambios remotos al repositorio local respetando las reglas.

Instrucciones:

4. Verifica en GitHub (remoto):

- Confirma que ambos cambios, el realizado directamente en GitHub y el de tu repositorio local, están reflejados en la rama dev.

Ejercicio 10: Fusionar ramas (Merge)

Objetivo:

Practicar cómo fusionar cambios entre ramas, respetando las reglas de protección establecidas en el repositorio.

Instrucciones:

1. En la rama dev (local):

- Crea un nuevo archivo o modifica uno existente, por ejemplo:
 - Archivo: nueva-funcionalidad.txt.
 - Contenido: " Funcionalidad desarrollada en dev."
- Haz un commit con el mensaje:
feat: añade nueva funcionalidad en dev
- Haz un **push** de los cambios al repositorio remoto en GitHub en la rama dev.

Ejercicio 10: Fusionar ramas (Merge)

Objetivo:

Practicar cómo fusionar cambios entre ramas, respetando las reglas de protección establecidas en el repositorio.

Instrucciones:

2.En GitHub (remoto):

- Crea un **Pull Request** desde dev hacia qa:
 - Ve al repositorio en GitHub y accede a la rama dev
 - Haz clic en “**Compare Pull Requests**”
 - Selecciona y verifica que **base: qa** (la rama donde se aplicarán los cambios) y **compare: dev** (la rama de donde provienen los cambios)
 - Describe un título : Por ejemplo : “Integración de cambios desde dev a qa”
 - Explica qué cambios estás integrando: “Este pull request incluye los siguientes cambios en la rama dev: - Actualización del archivo Readme.md , Añadido nuevas funcionalidades”

Ejercicio 10: Fusionar ramas (Merge)

Objetivo:

Practicar cómo fusionar cambios entre ramas, respetando las reglas de protección establecidas en el repositorio.

Instrucciones:

3.Simula la revisión en QA:

- Imagina que alguien revisa y aprueba los cambios.
- Haz clic en “**Create Pull Request**” tras la aprobación.
- Fusionar los cambios : Haz clic en "**Merge Pull Request**" en GitHub.

The screenshot displays a GitHub Pull Request interface. At the top, a list of commits is shown, including 'actualizar Readme desde Github', 'Porbar el merge', 'Update Readme.md', 'feat: cambios locales en dev', 'feat: añade nueva funcionalidad en dev', and 'Merge pull request #2 from Nagore189/dev'. Below this, a summary indicates 'Showing 2 changed files with 7 additions and 1 deletion.' The diff view for 'Readme.md' shows changes to the README file, and the diff view for 'nueva-funcionalidad.txt' shows the addition of a new file.

Ejercicio 10: Fusionar ramas (Merge)

Objetivo:

Practicar cómo fusionar cambios entre ramas, respetando las reglas de protección establecidas en el repositorio.

Instrucciones:

4. Haz clic en el botón “Corfirm Merge”

- Esto completará el proceso de fusión y aplicará los cambios de dev a qa

5. En Sourcetree (local):

- Cambia a la rama qa- Haz clic derecho sobre la rama qa en la lista de las ramas y selecciona “Checkout qa”
- Haz clic en el botón **Pull** en la barra superior.
- Asegúrate de que el origen del pull sea el remoto asociado a la rama qa (por ejemplo, origin/qa).
- Verifica que los cambios realizados y fusionados desde dev estén ahora en tu repositorio local en la rama qa.

La rama qa en local estará actualizada con los cambios fusionados desde el remoto. Esto completa el ciclo para asegurarte de que tanto el repositorio remoto como el local están sincronizados.

Ejercicio 10: Fusionar ramas (Merge)

Objetivo:

Practicar cómo fusionar cambios entre ramas, respetando las reglas de protección establecidas en el repositorio.

¿Qué hacer ahora?

1.Verifica los cambios en la rama qa:

- Abre la rama qa en GitHub y revisa los archivos para asegurarte de que los cambios esperados están presentes.
- Esto es importante para comprobar que el proceso de fusión se realizó correctamente.

2.Decidir los próximos pasos:

- Si los cambios han sido verificados como correctos en qa, puedes proceder a hacer otro Pull Request de qa a main para pasar los cambios a producción.
- Si aún necesitas hacer pruebas o validaciones en qa, continúa trabajando en esa rama antes de avanzar a main.

Ejercicio 11: Revertir un cambio

Objetivo: Aprender a deshacer un cambio sin eliminar el historial. (mi-proyecto)

Cambiar a la rama dev

1.En Sourcetree:

- Ve al panel lateral donde están las ramas (Branches).
- Busca y selecciona la rama dev.
- Haz clic derecho sobre la rama dev y selecciona Checkout dev.

2.Verifica que estás en la rama dev:

- En la barra superior de Sourcetree, deberías ver el nombre de la rama actual, por ejemplo: Current Branch: dev.

3. Realiza una modificación en el archivo README.md para introducir un error intencional:

- Abre el archivo README.md en tu editor de texto o IDE que quieras
- Introduce un cambio erróneo, como una línea incorrecta de texto, por ejemplo:

ERROR: Este es un cambio erróneo.

Ejercicio 11: Revertir un cambio

Objetivo: Aprender a deshacer un cambio sin eliminar el historial.

4. Haz un commit con el mensaje:

- En Sourcetree:
 - Asegúrate de que el cambio esté en estado "unstaged".
 - Haz clic en el botón **Stage Selected** (un ícono con una flecha hacia arriba) o arrastra el archivo a la sección **Staged Files**.
 - Haz clic en **Commit** y escribe el mensaje:
feat: añade error intencional

5. Realiza un push

- Una vez hecho el commit, haz clic en el botón **Push** en la barra superior de Sourcetree para enviar los cambios erróneos al repositorio remoto (GitHub) de la rama dev.
- Comprueba en GitHub que está ese commit erróneo subido.

Ejercicio 11: Revertir un cambio

Objetivo: Aprender a deshacer un cambio sin eliminar el historial.

En Sourcetree (local):

6. Selecciona el commit que contiene el error:

- Ve a la pestaña **History** en Sourcetree.
- Identifica el commit con el mensaje feat: añade error intencional.

7. Haz clic derecho y selecciona Reverse Commit:

- Esto creará un nuevo commit que revierte automáticamente los cambios introducidos en el commit erróneo.

8. Haz un push para reflejar el cambio revertido en el remoto:

- Una vez creado el commit de reversión, haz clic en **Push** para enviar este nuevo commit al remoto.

Resultado esperado:

- El error se deshizo mediante un nuevo commit que lo revierte.

Ejercicio 12: Crear un entorno colaborativo en Github

Objetivo: Configurar un repositorio colaborativo, establecer reglas y ramas, y trabajar en equipo.

Crear equipos de 2-3 personas.

Paso 1: Crear un repositorio en GitHub

1. Un miembro del equipo debe:

- Iniciar sesión en GitHub.
- Hacer clic en el botón **New Repository** (arriba a la derecha).
- Completar los campos:
 - **Repository Name:** nombre-del-repositorio.
 - **Description:** Opcional, algo como: "Repositorio colaborativo para prácticas".
 - Seleccionar **Public** para que sea accesible a todos los miembros.
- Activar la casilla **Initialize this repository with a README**.
- Hacer clic en **Create Repository**.

Ejercicio 12: Crear un entorno colaborativo en Github

Objetivo: Configurar un repositorio colaborativo, establecer reglas y ramas, y trabajar en equipo.

Paso 2: Configurar colaboradores

1.El creador del repositorio debe:

1. Ir a la pestaña **Settings** del repositorio.
2. Hacer clic en **Collaborators** (en el menú lateral izquierdo, dentro de **Access**).
3. Agregar los nombres de usuario de GitHub de los compañeros.
4. Los colaboradores recibirán un correo electrónico o notificación para aceptar la invitación.

Ejercicio 12: Crear un entorno colaborativo en Github

Objetivo: Configurar un repositorio colaborativo, establecer reglas y ramas, y trabajar en equipo.

Paso 3: Crear las ramas principales

El creador del repositorio debe:

1. Crear las ramas dev, qa y main:

- En la barra superior, haz clic en **main** → **New Branch**.
- Escribe dev como nombre de la rama y haz clic en **Create Branch**.
- Repite el proceso para qa.

Ejercicio 12: Crear un entorno colaborativo en Github

Objetivo: Configurar un repositorio colaborativo, establecer reglas y ramas, y trabajar en equipo.

2. Establecer reglas para proteger las ramas main y qa:

Ve a la pestaña **Settings** del repositorio.

En el menú lateral izquierdo, selecciona

Branches

Haz clic en **Add branch ruleset**.

The screenshot shows the GitHub 'New branch ruleset' configuration page. On the left is a sidebar menu with categories: General, Access (Collaborators, Moderation options), Code and automation (Branches, Tags, Rules, Rulesets, Actions, Webhooks, Environments, Codespaces, Pages), Security (Code security, Deploy keys, Secrets and variables), and Integrations (GitHub Apps, Email notifications). The 'Rulesets' option under 'Code and automation' is selected. The main content area is titled 'Rulesets / New branch ruleset'. It features a header 'Protect your most important branches' with a subtext explaining that rulesets define whether collaborators can delete or force push and set requirements for pushes. Below this is a 'Ruleset Name' field containing 'no direct merge'. The 'Enforcement status' is set to 'Active'. There is a 'Bypass list' section with a '+ Add bypass' button and a message 'Bypass list is empty'. The 'Targets' section asks 'Which branches do you want to make a ruleset for?' and shows 'Target branches' with a description: 'Branch targeting determines which branches will be protected by this ruleset. Use inclusion patterns to expand the list of branches under this ruleset. Use exclusion patterns to exclude branches.' Below this is a 'Branch targeting criteria' section with an 'Add target' button. It lists two targets: 'Default' and 'qa', each with a plus icon to add and a trash icon to remove.

Ejercicio 12: Crear un entorno colaborativo en Github

Objetivo: Configurar un repositorio colaborativo, establecer reglas y ramas, y trabajar en equipo.

2. Establecer reglas para proteger las ramas main y qa:

Configura las siguientes reglas:

Require a pull request before merging: Obliga a que los cambios en main sean aprobados mediante un Pull Request.

Require approvals: Solicita al menos 1 aprobación antes de fusionar cambios.

Require linear history

Activa esta opción para mantener un historial de commits limpio y lineal en main.

Haz clic en **Create** para guardar las reglas.

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 1: Cada persona del equipo clonará el repositorio en su máquina local

1. En GitHub (Remoto):

- Copia la URL del repositorio:
 - Haz clic en el botón verde **Code**.
 - Selecciona **HTTPS** y copia el enlace.

2. En cada máquina local:

- Abre Sourcetree o una terminal.
- Clona el repositorio:
 - Haz clic en **Clone**, pega la URL copiada, elige una carpeta local y haz clic en **Clone**.
 - Haz clic en el botón Fetch en la rama superior.
 - **Asegúrate de que la opción Fetch from all remotes esté seleccionada y haz clic en OK.**

Ejercicio 12.1: Sincronización y trabajo en equipo

Cada persona del equipo clonará el repositorio en su máquina local.

3. Verifica las ramas remotas:

- En el panel izquierdo, debajo de **REMOTES**, expande origin.
- Deberían aparecer las ramas remotas dev, qa, y main.
- **Trae las ramas remotas al local:**
 - Haz clic derecho sobre cada rama remota (origin/dev, origin/qa) y selecciona **Checkout**.
 - Esto creará las ramas locales correspondientes y las enlazará con las remotas.

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 2: Crear el archivo instrucciones.txt

1. Una persona del equipo crea el archivo:

- **Cambia a la rama dev (en local):**

- En Sourcetree, haz clic derecho sobre la rama dev y selecciona **Checkout dev**.
- Confirma que ahora estás trabajando en la rama dev.

- **Crea el archivo instrucciones.txt:**

- Abre el repositorio local en tu editor de texto (por ejemplo, VS Code o Notepad++).
- Dentro del repositorio, crea un archivo llamado instrucciones.txt.
- Escribe algunas instrucciones simples, como:

“Instrucciones para el ejercicio:

- 1. Realizar un pull.
- 2. Resolver conflictos si los hay.
- 3. Trabajar en equipo.”

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 2: Crear el archivo instrucciones.txt

1.Una persona del equipo crea el archivo:

Haz un commit del archivo:

- Vuelve a Sourcetree.
- Verás el archivo instrucciones.txt en la lista de cambios.
- Selecciona el archivo, escribe un mensaje de commit como:

feat: añade instrucciones para el ejercicio

Haz clic en **Commit**.

•Haz un push a la rama dev:

- Haz clic en el botón **Push** en la barra superior.
- Asegúrate de que estás enviando los cambios a la rama origin/dev y confirma.

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 3: Sincronizar los cambios

1. La otra persona del equipo sincroniza su repositorio local con los cambios remotos:

- **En Sourcetree:** Haz clic en **Pull** mientras estás en la rama dev.

2. Resultado esperado:

- Ambos estudiantes tienen el archivo instrucciones.txt en su repositorio local.

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 4: Generar un conflicto

1. Cada persona del equipo modifica el mismo archivo:

- Cada uno abre instrucciones.txt en su máquina local y realiza cambios diferentes. Por ejemplo:
 - Estudiante A: Añade una línea con "Nueva instrucción A".
 - Estudiante B: Añade una línea con "Nueva instrucción B".

2. Ambos realizan un commit y un push:

- En Sourcetree:
 - Arrastra los cambios a **Staged Files**.
 - Haz clic en **Commit** y escribe un mensaje, por ejemplo:
feat: añade cambios Haz clic en **Push**.

3. Se generará un conflicto para la segunda persona que intente hacer push.

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 5 Resolviendo el Conflicto

Cuando el **Estudiante B** intenta realizar un push, Sourcetree o Git notifica un conflicto porque el archivo instrucciones.txt ha sido modificado por ambos estudiantes. Este conflicto surge debido a cómo Git maneja el estado de HEAD.

¿Qué es HEAD en Git?

- En Git, **HEAD** representa el **punto de referencia actual** en tu repositorio.
- Apunta al último commit de la rama activa en tu máquina local.
- Cuando intentas realizar un push, Git compara tu HEAD local con el estado del remoto para verificar si hay conflictos.

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 5 Resolviendo el Conflicto

1. Estudiante B hace Pull:

- Haz clic en **Pull** en la rama dev para traer los cambios del remoto.
- Git intentará fusionar los cambios de la rama remota con los cambios locales del Estudiante B
- Sourcetree identificará un conflicto en instrucciones.txt.

2. Resolver el Conflicto:

- Abre el archivo instrucciones.txt en tu editor de texto.
- Git identifica las líneas en conflicto
- <<<<<< HEAD
- Nueva instrucción B
- =====
- Nueva instrucción A
- >>>>>> commit-id

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 5 Resolviendo el Conflicto

Estas marcas indican:

- HEAD: Los cambios locales del Estudiante B.
- commit-id: Los cambios remotos del Estudiante A.

Ejercicio 12.1: Sincronización y trabajo en equipo

Paso 5 Resolviendo el Conflicto

3- Marcar como resuelto:

- En Sourcetree, selecciona el archivo en conflicto y haz clic en **Mark Resolved**.

4- Commit de Resolución:

- Realiza un nuevo commit con el mensaje:

fix: resolver conflicto en instrucciones.txt

- Haz clic en Commit.

5- Actualiza el remoto

- Haz clic en **Push** para enviar los cambios al remoto.

Ejercicio 12.2: Trabajo con pull request

Paso 1: Realizar Cambios en la Rama dev

1.Un estudiante realiza cambios en su máquina local en la rama dev:

- Crea un archivo nuevo.
- Por ejemplo, añade esta línea en un archivo documentacion.txt:

Se añaden instrucciones adicionales para el equipo.

2.Realiza un **commit** con un mensaje claro:

feat: añadir instrucciones adicionales

3.Push:

- Sube los cambios al remoto (GitHub) en la rama dev.

Ejercicio 12.2: Trabajo con pull request

Paso 2: Crear un Pull Request desde dev hacia qa

1. Ese mismo estudiante entra en el repositorio en GitHub.
2. Cambia a la rama dev y hace clic en el botón verde "**Compare & pull request**".
3. Completa los siguientes campos:
 - **Título del Pull Request:** Actualización de instrucciones en documentacion.txt
 - **Descripción:** Documenta el propósito de los cambios, por ejemplo:
 - Qué se cambió.
 - Por qué se cambió.
 - Cómo afecta al proyecto.
 - Todo el equipo puede ver y entender los cambios realizados gracias a los comentarios y la documentación en el Pull Request.
 - **Pull Request permite a los miembros del equipo revisar los cambios antes de fusionarlos en una rama compartida.**
4. Haz clic en "**Create Pull Request**".

Ejercicio 12.2: Trabajo con pull request

Paso 3: Revisión del Pull Request

- 1.El compañero (estudiante B) entra en GitHub y revisa el Pull Request en la pestaña **"Pull Requests"**.
- 2.Verifica los cambios realizados:
 - Haz clic en el botón **"Files changed"** para ver exactamente qué se ha modificado. En esta vista, puedes revisar los cambios línea por línea.
- 3.Realiza comentarios si hay algo que discutir o mejorar:
 - En cada línea de código, puedes añadir un comentario haciendo clic en el "+".
 - Ejemplo de comentario:

¿Podrías añadir más detalles sobre las instrucciones?
- 4.Una vez revisado
 - Si estás conforme, aprueba los cambios desde el botón **"Review changes"** → **"Approve"**.
 - Si no estás de acuerdo, selecciona **"Request changes"** y escribe los ajustes necesarios.

Ejercicio 12.2: Trabajo con pull request

Paso 4: Fusión del Pull Request

1.Revisión final: (estudiante B)

1. Una vez que el compañero ha revisado y está de acuerdo con los cambios realizados, selecciona la opción **"Approve"** en el botón **"Review changes"**.

2.Fusión de los cambios: (estudiante A)

1. Haz clic en el botón verde **"Merge Pull Request"**.
2. Confirma la acción haciendo clic en **"Confirm Merge"**.
3. GitHub fusionará automáticamente los cambios de la rama **dev** a **qa**.

3.Verificación en la rama qa:

1. Después de la fusión, navega a la rama **qa** en GitHub y verifica que los cambios estén correctamente integrados.

Ejercicio 12.2: Trabajo con pull request

Paso 5:

Todos los miembros del equipo deben sincronizar sus repositorios locales para asegurarse de trabajar con la versión más reciente del proyecto.

Paso 1: El estudiante B sincroniza los cambios de qa (remoto a local)

1. Cambiar a la rama qa:

- En Sourcetree: Haz clic derecho sobre **qa** → **Checkout qa**.

2. Traer los cambios de qa desde el remoto:

- En Sourcetree: Haz clic en **Pull**.

Resultado esperado: Los cambios fusionados por el estudiante A en el remoto ahora están en el repositorio local del estudiante B.

Ejercicio 12.2: Trabajo con pull request

Paso 5:

Todos los miembros del equipo deben sincronizar sus repositorios locales para asegurarse de trabajar con la versión más reciente del proyecto.

Paso 2: Traer los cambios de qa a dev .

El estudiante B asegura que los cambios probados en qa también están en su copia local de dev

1.Cambiar a la rama dev:

- En Sourcetree: Haz clic derecho sobre **dev** → **Checkout dev**.

2.Fusionar los cambios de qa en dev (local):

- En Sourcetree: Haz clic derecho sobre **qa** → **Merge qa into dev**.

Resultado esperado: Los cambios de **qa** ahora también estarán en **dev**.

Ejercicio 12.2: Trabajo con pull request

Paso 5:

Todos los miembros del equipo deben sincronizar sus repositorios locales para asegurarse de trabajar con la versión más reciente del proyecto.

Paso 3: Sincronizar dev con el remoto

El estudiante B se asegura de que su copia local de **dev** esté actualizada con los cambios que el estudiante A haya subido al remoto.

1. Traer los cambios más recientes del remoto en dev:

- En Sourcetree: Haz clic en **Pull** mientras estás en la rama **dev**.

Resultado esperado: Ahora, tanto los cambios locales como los cambios remotos en **dev** estarán sincronizados.

